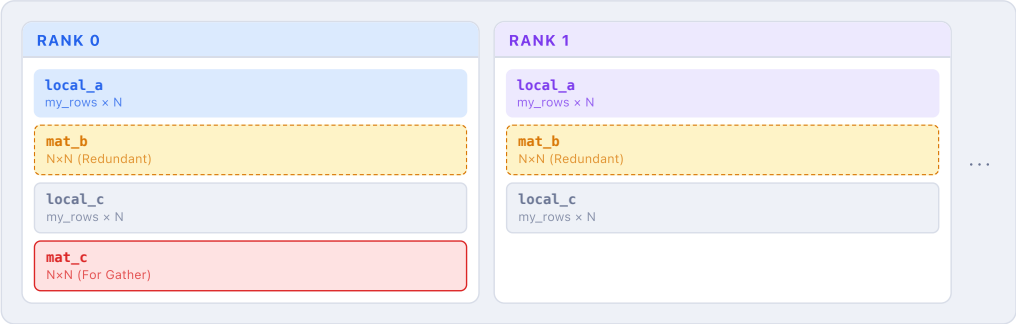
Slide 2: Memory Management& Initialization

Speaker: Md Fuzail

Streamlined Memory Map

local_a mat_b

No intermediate slices. Memory is allocated strictly for computation.



matmul.c — initialization

```c
/* EVERY rank generates full matrix B locally */
double *mat_b = alloc_matrix(n, n);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(i, j) + seed + 1;
        mat_b[i * n + j] = my_rand(&state, 0, 1);
    }
}

/* Each rank generates its specific A rows using offset */
double *local_a = alloc_matrix(my_rows, n);
for (int i = 0; i < my_rows; i++) {
    int global_row_idx = my_row_start + i;
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(global_row_idx, j) + seed + 0;
        local_a[i * n + j] = my_rand(&state, 0, 1);
    }
}
```

Data Structures

local_a — DISTRIBUTED

Only the slice needed is allocated: `my_rows × N`. Scales perfectly with node count.

mat_b (Full) — REDUNDANT GEN

No intermediate `slice` buffers. The full N×N matrix is generated straight into RAM using local CPU cycles.

local_c — RESULT BUFFER

Pre-initialized to zero using `memset`. Stores the local partial product of the multiplication.

1D Contiguous Allocation

All matrices use flat 1D arrays (`malloc(rows * cols)`). This prevents memory fragmentation, eliminates pointer-chasing, and guarantees cache locality.

Zero-Comm Strategy

Network Bypass

By abandoning collective communication completely, the cluster network bandwidth consumption is strictly **zero** during startup.

Offset Math

Nodes use `my_row_start + i` to independently seed the RNG. This guarantees all nodes compute the exact correct slice of Matrix A.

Constraint: Master Memory

Rank 0 must allocate a massive final `mat_c` (N×N) array just before `MPI_Gatherv` to receive and checksum the data.

MEMORY / NODE	INIT TIME
$O(N^2/P + N^2)$	$O(N^2)$
slice + full B replica	Fixed redundant CPU cost